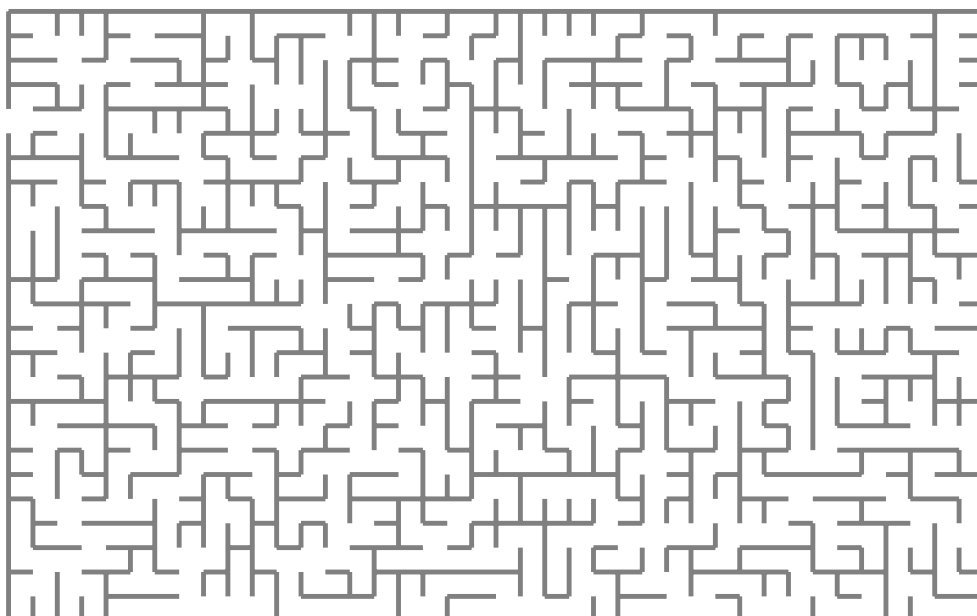


Higher order pattern unification using scope graphs

Version of March 25, 2026



Tudor Andrei

Higher order pattern unification using scope graphs

THESIS

submitted in partial fulfillment of the
requirements for the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE

by

Tudor Andrei
born in Bucharest, Romania



Programming Languages Group
Department of Software Technology
Faculty EEMCS, Delft University of Technology
Delft, the Netherlands
www.ewi.tudelft.nl

© 2026 Tudor Andrei.

Cover picture: Random maze.

Higher order pattern unification using scope graphs

Author: Tudor Andrei
Student id: 5495822
Email: T.Andrei@student.tudelft.nl

Abstract

This work researches the possibility of using higher order unification in a typechecker that uses scope graphs. Scope graphs are a novel data structure for name resolution created by Statix. The Statix language is part of the language workbench Spoofax, created at TU Delft, and enables users to write a typechecker for their language. Out of the box, Statix can do first order unification between terms from the user's language which is enough to typecheck the majority of type system. However, dependently typed languages present a bigger challenge, which arises from type checking definitions with implicit arguments or pattern matching. We will propose a new unification algorithm for Statix, that is able to unify higher order terms (lambda functions) while using scope graphs for name resolution.

Thesis Committee:

Chair:	Prof. dr. C. Hair, Faculty EEMCS, TU Delft
Committee Member:	Dr. A. Bee, Faculty EEMCS, TU Delft
Committee Member:	Dr. C. Dee, Faculty EEMCS, TU Delft
University Supervisor:	Ir. E. Ef, Faculty EEMCS, TU Delft

Preface

Preface here.

Tudor Andrei
Delft, the Netherlands
March 25, 2026

Contents

Preface	iii
Contents	v
List of Figures	vii
List of Tables	ix
1 Introduction	1
1.1 Contributions	2
2 Background: Statix	5
2.1 Parsing the AST	5
2.2 Unification	6
2.3 Statix	6
2.4 Scope Graphs	7
3 Results	9
4 Related work	11
5 Conclusion	13
Bibliography	15
Acronyms	17
A A	19

List of Figures

3.1	Inference rules	9
3.2	Checking rules	10

List of Tables

Chapter 1

Introduction

Having a strong type system has become a defining characteristic of many modern programming languages. Beyond basic error detection, expressive type systems allow developers to encode invariants directly into the structure of their programs, shifting entire classes of bugs from runtime to compile time. Types serve as a precise form of documentation, making assumptions and guarantees explicit and enforceable.

Dependent type systems represent a significant step in this progression. Positioned at the top of the lambda cube, they extend conventional type systems by allowing types to depend on values. This added expressiveness enables the encoding of precise specifications. For instance, types that capture the exact length of a list or enforce domain-specific constraints. Besides the regular function type $A \rightarrow B$, this calculus introduces Π -types of the form: $(x : A) \rightarrow B$, where x can appear in the definition of B . x is only known when we apply the function on an argument. The actual value of the argument should be substituted for x in B . Additionally, through the Curry-Howard correspondence (Reis n.d.), such type systems blur the lines between programs and proofs: writing a well-typed program becomes equivalent to constructing a proof of a logical proposition. As a result, languages with dependent types can function as proof assistants for propositional logic with quantifiers. Practical realizations of these ideas can be seen in systems such as Lean, Agda, and Idris 2, where programming and formal verification are tightly integrated. For a dependently typed language to be considered practical, it needs to perform as much type inference as possible. Consider the function to add an element to the head of a list in a dependent type setting where all types are explicit:

$$\text{cons} :: (A : \text{Set}) \rightarrow (n : \text{Int}) \rightarrow A \rightarrow \text{Vec } A \ n \rightarrow \text{Vec } A \ (n + 1)$$

cons is polymorphic with respect to the type of the list and the length of the list. Providing an explicit argument for the type and length of the list everytime the function is called is very cumbersome. To this end, production implementations manage to infer some of the types and values, making them implicit (they appear in the signature but can be omitted on application). Inferring implicit arguments is a big challenge for dependent type checkers and requires higher order (pattern) unification (Johansson and Lloyd n.d.) between terms that can be either values or types.

Language workbenches are environments designed to support the systematic development of programming languages (Erdweg et al. 2015) by providing reusable infrastructure for common compiler and tooling concerns. They exist primarily to reduce the engineering overhead associated with implementing languages from scratch, particularly for very common features such as parsing, name binding, type checking, and editor support. By elevating these concerns to declarative specifications, language workbenches allow language designers to focus on semantics rather than implementation details. Language workbenches enable rapid prototyping, facilitate experimentation with advanced type systems, and promote correctness through well-defined meta-languages and tooling. As an example, in the

case of dependent type languages, building all the necessary tooling from scratch is a difficult task. Type checking in such systems often involves combining mechanisms like higher-order unification, weak head normalization and careful handling of variable binding and scope. Language workbenches provide abstractions that address these challenges. Core concerns such as name resolution, binding structure, and unification are treated as first-class components, allowing language designers to focus on the semantics of their type systems rather than low-level implementation details.

Spoofax (Wachsmuth, Konat, and Visser 2014), is a language workbench developed at TU Delft. Type checking in Spoofax is done using Statix (Antwerpen et al. 2018), a constraint-based language designed for expressing typing and name resolution rules declaratively. The main contribution of Statix is name resolution through scope graphs, which are graph-based representation of scopes and the relations between them. Scope graphs are particularly well suited for imports, mutual references and other types of non-lexical scoping. Since scope graphs are used to model scoping, it is not possible to create function terms in Statix. Likewise, the semantics of substitution must be explicitly encoded by the programmer.

On the other hand, other approaches to name binding have been studied. Name binding can be represented more closely to lambda calculus via higher order abstract syntax (HOAS) (Pfenning and Elliott n.d.). The core idea of HOAS is that functions (lambda abstractions) are first class terms. They can be created by the user and they can also be deconstructed. A system implementing HOAS will give the user the ability to go under binders, by generating fresh placeholders for the bound name. Besides generating fresh names, a HOAS system will provide the implication operator that acts as an implicit context. An expression of the form $P \Rightarrow F$ can be viewed as “If predicate P holds, then prove F ”. This paradigm can be used for typechecking, to emulate typing contexts very closely to their formal definition: “Assuming that variable X has type A , prove the body has type B ”.

HOAS and Scope Graphs differ fundamentally in how they treat binding and unification. Since HOAS follows the syntax of simply typed lambda calculus, names are bound only by constructing a lambda abstraction while properties are asserted via implication. In return, users get capture-avoiding substitution, lexical scoping and higher order unification for free. While scope graphs provide strong support for modular and non-lexical scoping, they do not natively provide higher-order pattern unification, which is central to dependently typed systems.

This friction motivates the central question of this thesis: can higher-order pattern unification be integrated with a scope-graph-based constraint system in a principled and practical way? Furthermore, how does such an approach compare to higher-order logic programming systems when implementing a dependently typed language?

To address these questions, we design and evaluate a higher-order pattern unification algorithm tailored to a scope-graph-based setting, and analyze its implications for implementing dependently typed languages in Spoofax.

1.1 Contributions

This thesis addresses two research questions and makes the following contributions.

Higher-order pattern unification in scope-graph systems (RQ1). We design and formalize an algorithm for higher-order pattern unification in a constraint-based system that uses scope graphs for name binding. To validate the algorithm, we implement a minimal dependently typed core language in Statix, demonstrating that higher-order pattern unification can be integrated with scope-graph-based name resolution and supports core dependent type features.

Comparison of implementation paradigms for dependent typing (RQ2). We provide a structured comparison between higher-order logic programming systems such as λ -Prolog and scope-graph-based constraint systems. The analysis examines differences in expressivity and computability, including the treatment of binding, fresh name generation, modularity, and non-lexical scoping. This clarifies the trade-offs between meta-level binding mechanisms and explicit scope representations when implementing practical dependently typed languages.

Chapter 2

Background: Statix

This chapter will focus on the concepts behind Statix. We also think it is necessary to offer readers information about Spoofax and its features. It is important to understand where Statix sits in the context of the Spoofax framework to get a better insight into what's possible to do in a typechecker.

Spoofax is a framework that allows users to easily implement end-to-end tooling for their programming language. Made primarily for use within Eclipse, Spoofax comes as a plugin. In Spoofax, users start by defining the grammar of the language using SDF3. Later, one can write various passes, such as desugaring steps, IDE services or an interpreter, on the AST with Stratego and typing rules using Statix.

2.1 Parsing the AST

Getting an AST representation of the source code is one of the first things needed to compile or typecheck a language. This process gives meaning to the tokens found in a source code file. In SDF3, the parsing rules are defined declaratively, similarly to BNF notation. You can define context-free sorts with multiple constructors. The constructors can match on code tokens or other sorts. As an example, users might find it helpful to define a sort for expressions and another for types. The parsing starts from a top level sort and will try to match the tokens with the defined constructors. As an example consider the grammar in figure.

```
context-free sorts
  Expr
  Type

context-free syntax
  Expr.Num = Number
  Expr.Eq = [[Expr] == [Expr]] {left}
  Expr.Add = [[Expr] + [Expr]] {left}
  Expr.Var = Name
  Expr.Lam = [[Name] : [Type] -> [Expr]]
  Expr.App = [[Expr] [Expr]] {right}

  Type.Nat = [nat]
  Type.Fun = [[Type] -> [Type]]

context-free priorities
  {Expr.Num Expr.Var} > Expr.App > Expr.Add > Expr.Eq > Expr.Lam
```

Listing 2.1: Simple language with Type and Expr sorts

The AST is in **ATerm** format and will contain the constructors that were matched. For instance, the program `\x : Int -> x + 1`, will produce the AST:

```
Expr.Lam("x", Type.Nat(), Expr.Add(Expr.Var("x"), Number(1)))
```

2.2 Unification

This section will explain what unification is, where it is used and the different types of algorithms.

In general unification is concerned with filling in missing parts of terms in order to make them equal. Unification wants to know if two terms A and B are equal, and if not, if they can be made equal by filling in their free variables. For example, consider the unification problem $f(x, y) \equiv f(1, g(2))$ (assume f and g are known functions). The algorithm should find the substitutions $x \mapsto 1, y \mapsto g(2)$.

Common requirements for a unification algorithm are decidability and the generality of the solution. Decidability is a natural requirement, so it is interesting to discuss what generality of the solution means. We would prefer if our algorithm will find the substitution that doesn't constraint the term more than it needs to. Consider the problem of unifying the terms $f(x, y) \equiv f(y, x)$. A possible solution is the substitution $x \mapsto 1, y \mapsto 1$. However this solution constraints the free variables too much. The most general solution to this problem is $x \mapsto y$. Given the difficulty of unification, one may wonder if a decidable algorithm exists for the most general unifier (MGU). To answer this, we must make a distinction on the type of terms we will substitute for the free variables. The biggest distinction is between so called first order and higher order terms. First order terms are variables, constants and in general terms that don't bind new variables. First order unification is decidable and admits the most general unifier.

2.3 Statix

Statix is used by Spoofox to perform typechecking of the user's language. Statix is based on terms and unification. The main concepts of Statix are:

- **Terms:** This is the smallest abstraction that Statix provides. Statix terms can be unified using the first order unification algorithm built into the language.
- **Rules:** These are similar to predicates in Prolog. Given an instantiation of a rule, Statix will try to prove it. Rules can pattern match on their arguments, and in each case will require some constraint(s) to hold in order for them to be proven.
- **Constraints:** Constraints are problems that Statix will work on involving term and scope graph operations. Possible constraints are (dis)equality, conjunction, disjunction and scope graph constraints. For example, a rule can require that two terms are equal, or that a new scope is added to the scope graph.

Terms in Statix follow a sorted logic. The user can declare new sorts and constructors for objects in the sort. This design differs from λ -Prolog as you cannot create lambda terms. They are not first class citizens in Statix. Moreover, Statix does not perform function application on terms.

Rules are the Statix equivalent of predicates in λ -Prolog. In λ -Prolog, the convention is that the output of a predicate is stored in its last argument. The caller will provide a fresh, unfilled variable to the predicate. When the predicate is proven, the variable will be filled with a value through unification. This is also the case in Statix. However, Statix also

You can introduce variables in the rule head, but at runtime it will receive an actual value. Variables introduced as existential will be filled by unification

offers functional rules, which mimic functions in functional programming languages. Functional rules are converted by Statix to regular rules as syntactic sugar. When simplifying constraints, Statix doesn't backtrack. This is different to λ Prolog implementations such as Elpi(). Statix will try at most one rule for each constraint. The appropriate rule is selected by applying the following heuristics in order: 1. Rules with a smaller domain are preferred over rules with a larger domain. 2. When pairwise comparing rules, the rule for which, in left-to-right order, a more specific pattern is encountered first is preferred over the other. For all the cases where the heuristic can't decide, Statix emits compile time errors.

Although the Statix language does not distinguish between variables in rule heads and existentials, the solver treats those quite differently. On rule application, occurrences of variables from rule heads in the body of the rule are substituted by the actual arguments. For example, in a specification containing the rule $\text{rule}(x) :- x == ()$., simplifying the constraint $\text{rule}()$ will result in the residual constraint $() == ()$, which trivially holds. However, when simplifying the constraint $x x == ()$, simplifying will generate a fresh unification variable (say $?x-1$), and substitute that in the constraint, yielding $?x-1 == ()$. Solving that will create a new mapping $?x-1 \mapsto ()$ in the unifier of the solver result.

2.4 Scope Graphs

Chapter 3

Results

To demonstrate the combination of scope graph name resolution with higher order unification, we will describe how to typecheck a dependent type language. The main feature of this language is the ability to infer implicit arguments.

AB	$::= Set \mid A \equiv B \mid A \rightarrow B \mid (x : A) \rightarrow B \mid \{x : A\} \rightarrow B$	<i>types</i>
MN	$::= x \mid ?m \mid M :: A \mid \lambda x. M \mid M N \mid refl \mid let x = M; N \mid postulate x : A; M$	<i>terms</i>
Γ	$::= \Gamma; x : A$	<i>typing context</i>
Δ	$::= \Delta; ?m : A$	<i>meta – context</i>

Typechecking is done with bidirectional rules with $\Rightarrow$ for inferring Figure 3.1 and $\Leftarrow$ for checking Figure 3.2. For some rules the relation $A = B$ appears, which suggests that A and B should be unified. The meta context Δ should have mutable semantics, meaning that solving or creating a metavariable in one branch of the inference tree should mutate the context for all the other branches. This is opposed to the typing context, which is responsible for storing the identifiers that are in scope, associated with their types.

$\frac{\text{META} \quad ?a : A \in \Delta}{\Gamma, \Delta \vdash ?a \Rightarrow A}$	$\frac{\text{VAR} \quad x : A \in \Gamma}{\Gamma, \Delta \vdash x \Rightarrow A}$	$\frac{\text{SET}}{\Gamma, \Delta \vdash \text{Set} \Rightarrow \text{Set}}$
$\frac{\text{PI} \quad \Gamma, \Delta \vdash t \Rightarrow (x : A) \rightarrow B \quad \Gamma, \Delta \vdash u : A}{\Gamma, \Delta \vdash t u \Rightarrow B[x/u]}$		
$\frac{\text{IMPPi} \quad \Gamma, \Delta \vdash t \Rightarrow \{x : A\} \rightarrow B \quad \Delta; ?a \vdash (t ?a) u \Rightarrow C}{\Gamma, \Delta \vdash t u \Rightarrow C}$	$\frac{\text{LET} \quad \Gamma, \Delta \vdash t \Rightarrow A \quad \Gamma; x : A, \Delta \vdash u : B}{\Gamma, \Delta \vdash \text{let } x = t; u \Rightarrow B}$	
$\frac{\text{POSTULATE} \quad \Gamma, \Delta \vdash A \Rightarrow \text{Set} \quad \Gamma; d : A, \Delta \vdash u : B}{\Gamma, \Delta \vdash \text{postulate } d : A; u \Rightarrow B}$		

Figure 3.1: Inference rules

$$\begin{array}{c}
\text{LAM} \\
\frac{\Gamma; x : A, \Delta \vdash t \Leftarrow B}{\Gamma, \Delta \vdash \lambda x. t \Leftarrow (x : A) \rightarrow B} \\
\\
\text{SET} \\
\frac{}{\Gamma, \Delta \vdash \text{Set} \Leftarrow \text{Set}} \\
\\
\text{IMPLICIT} \\
\frac{\Gamma, \Delta; ?m : A \vdash t \Leftarrow B[x/?m]}{\Gamma, \Delta \vdash t \Leftarrow \{x : A\} \rightarrow B} \\
\\
\text{REFL} \\
\frac{\Gamma, \Delta \vdash X = Y}{\Gamma, \Delta \vdash \text{refl} \Leftarrow X \equiv Y} \\
\\
\text{PI-WELLFORMED} \\
\frac{\Gamma, \Delta \vdash A \Leftarrow \text{Set} \quad \Gamma; x : A, \Delta \vdash B \Leftarrow \text{Set}}{\Gamma, \Delta \vdash (x : A) \rightarrow B \Leftarrow \text{Set}} \\
\\
\text{IMPI-WELLFORMED} \\
\frac{\Gamma, \Delta \vdash A \Leftarrow \text{Set} \quad \Gamma; x : A, \Delta \vdash B \Leftarrow \text{Set}}{\Gamma, \Delta \vdash \{x : A\} \rightarrow B \Leftarrow \text{Set}} \\
\\
\text{EQ-WELLFORMED} \\
\frac{\Gamma, \Delta \vdash t \Rightarrow A \quad \Gamma, \Delta \vdash u \Rightarrow B \quad A = B}{\Gamma, \Delta \vdash t \equiv u \Leftarrow \text{Set}} \\
\\
\text{INFER} \\
\frac{\Gamma, \Delta \vdash t \Rightarrow A' \quad A = A'}{\Gamma, \Delta \vdash t \Leftarrow A}
\end{array}$$

Figure 3.2: Checking rules

3.0.1 Meta-context in scope graph

To model the meta-context in the scope graph framework we instantiate a toplevel meta-scope called `metaS`. Each rule should be given a reference to this exact scope, such that it has permission to extend it. To model the creation of new unsolved metavariable we create a new scope s' with an M labeled edge to `metaS`. The reference to the scope s' becomes the unique identifier of the metavariable. When we want to instantiate a metavariable, we will create the `inst[id, V]` relation in `metaS`, where `id` is the metavariable's identifier and `V` represents its value. Such a relation can be easily queried for in any branch of our inference. Since we always have a reference to `metaS`, we want to search for `inst` relations with a given `id`, without leaving `metaS`

Chapter 4

Related work

Related work here.

Chapter 5

Conclusion

Conclusion here.

Bibliography

- Antwerpen, Hendrik van et al. (Oct. 24, 2018). “Scopes as types”. In: *Case Studies for Article: Scopes as Types 2 (OOPSLA)*, 114:1–114:30. doi: 10.1145/3276484. URL: <https://dl.acm.org/doi/10.1145/3276484> (visited on 01/09/2026).
- Erdweg, Sebastian et al. (Dec. 1, 2015). “Evaluating and comparing language workbenches: Existing results and benchmarks for the future”. In: *Computer Languages, Systems & Structures*. Special issue on the 6th and 7th International Conference on Software Language Engineering (SLE 2013 and SLE 2014) 44, pp. 24–47. ISSN: 1477-8424. doi: 10.1016/j.cl.2015.08.007. URL: <https://www.sciencedirect.com/science/article/pii/S1477842415000573> (visited on 03/23/2026).
- Johansson, Marcus and Jesper Lloyd (n.d.). “Eliminating the problems of hidden-lambda insertion”. In: ().
- Pfenning, Frank and Conal Elliott (n.d.). “Higher-Order Abstract Syntax”. In: ().
- Reis, Giselle (n.d.). “Curry-Howard Correspondence”. In: ().
- Wachsmuth, Guido H., Gabriël D.P. Konat, and Eelco Visser (2014). “Language Design with the Spoofox Language Workbench”. In: *IEEE Software* 31.5, pp. 35–43. doi: 10.1109/MS.2014.100.

Acronyms

AST abstract syntax tree

DSL domain-specific language

Appendix A

A

Appendix here.